Programming
Human-Centered Artificial Intelligence
Dr. Ramon Winterhalder
A.Y. 2026/27

Exercise sheet 4 — Solutions

Iterations – while, for, and the patterns inside every loop

Published after the in-class discussion. Read these only once you have tried the sheet.

Part A — while

A1 *Trace four loops*

1 — `total` is `6`, `i` is `4`.

pass	`i`	`i < 4`	`total`
before	0	—	0
1	0	True	0
2	1	True	1
3	2	True	3
4	3	True	6
stop	4	False	6

Note that adding 0 on the first pass does nothing. The loop runs four times and the sum is $0+1+2+3$.

2 — `n` is `1`, `count` is `4`. The sequence is $20 \to 10 \to 5 \to 2 \to 1$. Note $5//2 = 2$: floor division, as in lecture 2. Counting how many times you can halve a number before reaching 1 is a question that comes back in lecture 10.

3 — `result` is `120`, `x` is `0`. It computes 5!, the factorial. Anyone who recognised it before running it has understood the accumulator.

4 — `i` is `12`. The values are $3, 6, 9, 12$; at 12 the condition `i > 10` is true and `break` leaves immediately. Note the loop exits at 12, not 11 — `break` happens after the change, not before.

A2 *Never starts, never stops, or runs*

	verdict	why
1	never starts	`i` is already 10, so `i < 5` is False at the first check
2	never stops	no change: `i` stays 1 for ever
3	runs 4 times	prints 1, 2, 3, 4 — stops when `i` reaches 5
4	**never stops**	see below
5	runs 3 times	5, 3, 1 — then `i` is −1 and the condition fails
6	runs 4 times	3, 2, 1, 0 — >= includes zero

Number 4 is the one worth discussing. It has a change, and the change moves `i` upwards, so it looks right. But `i` takes the values $0, 3, 6, 9, 12, 15, \ldots$ and *never equals 10*. The condition `i != 10` is therefore true for ever.

Worth noticing. The lesson: test with < or >, not `!=`, when a counter moves in steps. `while i < 10:` would have stopped at 12 — late, but stopped. `!=` demands that the counter land exactly on the target, and with a step of 3 it never does. The same trap is waiting for you with floats, where landing exactly on a value is almost impossible.

A3 *The accumulator*

```
# ask for n
# a running total, starting at 0
# a counter, starting at 1
# while the counter has not passed n:
#     add it in, move the counter on
# print the total

n = int(input("n: "))

total = 0
i = 1

while i <= n:
    total = total + i
    i = i + 1

print(f"The sum of 1 to {n} is {total}")
```

For $n = 0$ it prints `0`, because the condition `1 <= 0` is False and the body never runs.

That is defensible — the sum of no numbers is 0 — and it is worth noticing that the program never decided this. It fell out of the loop condition. A program that happens to be right is not the same as a program that is right on purpose.

Worth noticing. This is exactly sheet 1, A3(c), which asked the same question about the same algorithm in pseudocode. Then you could only reason about it. Now you can run it — and the answer has not changed.

A4 *Keep asking until it is valid*

Asked twice:

```
grade = int(input("Grade out of 30: "))

while grade < 0 or grade > 30:
    print("That is not between 0 and 30.")
    grade = int(input("Grade out of 30: "))

print(f"Grade recorded: {grade}")
```

With `break`:

```
while True:
    grade = int(input("Grade out of 30: "))
    if 0 <= grade <= 30:
```

```
        break
    print("That is not between 0 and 30.")

print(f"Grade recorded: {grade}")
```

Both are correct and both are used in practice. The first duplicates the question, so a later change has to be made twice. The second asks it once, at the cost of a condition that is always true and an exit hidden in the middle.

The second version also let us write the condition the positive way round (`0 <= grade <= 30`, "it is valid") instead of the negative (`grade < 0 or grade > 30`, "it is not valid"), which most people find easier to read. That, rather than the line count, is the better argument for it.

A5 *It runs, and the answer is wrong*

a) It prints `5`.

b) It should print `15`.

c) `total = 0` is *inside* the loop, so the total is wiped at the start of every pass. Each pass therefore computes $0 + i$, and what survives is the last one: $0 + 5 = 5$.

That is why the answer is 5 rather than something obviously absurd — and it is exactly what makes the bug hard to spot. With $n = 1$ the program is even correct.

d) Move `total = 0` above the loop:

```
total = 0
i = 1
while i <= 5:
    total = total + i
    i = i + 1
```

Worth noticing. Declared outside, updated inside, read after. Every accumulator has this shape, and this bug is what happens when you get it wrong. Look for it whenever a loop gives you the answer for the *last* item instead of all of them.

Part B — for and range

B1 *What does range give you?*

	written	gives
1	`range(5)`	0 1 2 3 4
2	`range(2, 7)`	2 3 4 5 6
3	`range(0, 10, 3)`	0 3 6 9
4	`range(5, 0, -1)`	5 4 3 2 1
5	`range(3, 3)`	nothing — start and stop are equal
6	`range(10, 0)`	nothing — counting up from 10 never reaches 0

Number 6 needs a negative step to do what it looks like it wants: `range(10, 0, -1)`. Without one it is an empty loop, and an empty loop raises nothing — it just silently does not happen.

The numbers 1 to `n` inclusive:

```
for i in range(1, n + 1):
```

B2 *Nested loops*

1 — 6 lines. 3×2. First three: `0 0`, `0 1`, `1 0`.

2 — 9 lines. The outer loop prints one line itself and the inner prints two, so each outer pass produces three lines. First three: `row 0`, `col 0`, `col 1`.

3 — 3 lines: `*`, `**`, `***`. `"*" * i` is string repetition, from sheet 2, A1.

4 — 2 lines: `0` and `0 1`. The inner loop runs `i + 1` times, so the rows get longer.

If the bare `print()` were indented one level further it would sit inside the inner loop, ending the line after every single number. The output would become four lines, one number each — the shape collapses.

Worth noticing. That last question is the whole of nested-loop indentation in one move. Which loop a statement belongs to is decided by how far right it sits, and nothing else.

B3 *Off by one*

	what it actually does	fix
1	sums 1 to $n-1$; for $n = 5$ gives 10 instead of 15	`range(1, n + 1)`
2	counts 10 down to 2 — never prints 1	`range(10, 0, -1)`
3	prints four stars	`range(5)` or `range(1, 6)`

All three are the same mistake: the stop value is *not* included, and each one was written as though it were.

For number 3, `range(5)` is the better fix: the counter is never used inside the loop, so starting at 0 costs nothing and there is no `+1` to get wrong.

B4 *Two short ones*

```
n = int(input("Count down from: "))

for i in range(n, 0, -1):
    print(i)

print("Liftoff")
```

```
n = int(input("Times table for: "))

for i in range(1, 11):
    print(f"{n} x {i} = {n * i}")
```

The countdown is the natural home for a negative step. Writing it with `while` is perfectly possible and takes three more lines, each of which can be got wrong.

B5 *Factorial*

```
n = int(input("n: "))
```

```
result = 1

for i in range(1, n + 1):
    result = result * i

print(f"{n}! = {result}")
```

a) The accumulator starts at **1**, not 0, because the operation is multiplication: 1 is the value that leaves things unchanged, the way 0 does for addition. Starting at 0 gives 0 for every input, which is a memorable way to learn this.

b) $5! = 120$. For $n = 0$ the loop is `range(1, 1)`, which is empty, so the body never runs and `result` is still 1 — which is the correct value of 0!.

Worth noticing. The program gets 0! right without a special case, because the starting value of an accumulator *is* the answer for an empty loop. That is not a coincidence, and it is a good reason to think about the starting value rather than copying it.

B6 *Shapes*

```
n = int(input("Height: "))

for i in range(1, n + 1):
    print("*" * i)
```

The pyramid:

```
n = int(input("Height: "))

for i in range(1, n + 1):
    spaces = n - i
    stars = 2 * i - 1
    print(" " * spaces + "*" * stars)
```

The two formulas are the exercise. Row i needs $n - i$ spaces (so the first row is pushed furthest right and the last not at all) and $2i - 1$ stars (1, 3, 5, 7 — always odd, which is what makes a centre possible).

Students who tried to write the code first will have spent a long time adjusting numbers by trial and error. Those who drew four rows on paper and counted got both formulas in two minutes. That difference is the point of the exercise, and it is worth saying so.

B7 *The largest*

```
how_many = int(input("How many numbers? "))

largest = int(input("Number 1: "))     # the first one is the best so far
count = 1

for i in range(2, how_many + 1):
    value = int(input(f"Number {i}: "))
    if value > largest:
        largest = value
        count = 1
    elif value == largest:
        count = count + 1
```

```
print(f"Largest: {largest}")
print(f"It appeared {count} time(s)")
```

a) `largest` starts as **the first number read**, not as 0.

Starting at 0 looks reasonable and is wrong: if every number typed is negative, 0 is larger than all of them and the program reports 0 — a number the user never entered. Starting with a real value from the data cannot fail.

c) The count resets to 1 whenever a new maximum is found, and increases only on a tie. Doing it in one pass requires this; doing it in two passes (find the largest, then count the matches) is easier to get right and needs the numbers kept somewhere, which is lecture 6.

Worth noticing. This is sheet 1, worked example 2 — "assume one is the winner, challenge it with each of the others" — now written as a program. The shape is the accumulator with "bigger than" in place of "plus". Recognising that two problems have the same shape is most of what experience in programming actually is.

If you finish early

Second largest in one pass needs two variables updated together, and the order matters:

```
if value > largest:
    second = largest        # the old champion is now second
    largest = value
elif value > second:
    second = value
```

It is easy to get subtly wrong — try it with all-equal numbers, or with only one number.

With a list you would write `sorted(numbers)[-2]` and be finished. That is lecture 6, and this exercise is the argument for it.